

A COMPUTATIONAL INTRODUCTION TO DIGITAL IMAGE PROCESSING

S E C O N D E D I T I O N

Alasdair McAndrew

Victoria University
Melbourne, Australia



CRC Press

Taylor & Francis Group

Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business
A CHAPMAN & HALL BOOK

MATLAB® is a trademark of The MathWorks, Inc. and is used with permission. The MathWorks does not warrant the accuracy of the text or exercises in this book. This book's use or discussion of MATLAB® software or related products does not constitute endorsement or sponsorship by The MathWorks of a particular pedagogical approach or particular use of the MATLAB® software.

CRC Press
Taylor & Francis Group
6000 Broken Sound Parkway NW, Suite 300
Boca Raton, FL 33487-2742

© 2016 by Taylor & Francis Group, LLC
CRC Press is an imprint of Taylor & Francis Group, an Informa business

No claim to original U.S. Government works
Version Date: 20150824

International Standard Book Number-13: 978-1-4822-4733-6 (eBook - PDF)

This book contains information obtained from authentic and highly regarded sources. Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, please access www.copyright.com (<http://www.copyright.com/>) or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. CCC is a not-for-profit organization that provides licenses and registration for a variety of users. For organizations that have been granted a photocopy license by the CCC, a separate system of payment has been arranged.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

Visit the Taylor & Francis Web site at
<http://www.taylorandfrancis.com>

and the CRC Press Web site at
<http://www.crcpress.com>

To my dear wife, Felicity, for all her love, support and understanding during my writing of
this book.

Contents

Preface	ix
1 Introduction	1
1.1 Images and Pictures	1
1.2 What Is Image Processing?	1
1.3 Image Acquisition and Sampling	4
1.4 Images and Digital Images	8
1.5 Some Applications	10
1.6 Image Processing Operations	11
1.7 An Image Processing Task	11
1.8 Types of Digital Images	12
1.9 Image File Sizes	15
1.10 Image Perception	16
Exercises	18
2 Images Files and File Types	19
2.1 Opening and Viewing Grayscale Images	19
2.2 RGB Images	22
2.3 Indexed Color Images	24
2.4 Numeric Types and Conversions	28
2.5 Image Files and Formats	30
2.6 Programs	38
Exercises	40
3 Image Display	43
3.1 Introduction	43
3.2 The <code>imshow</code> Function	44
3.3 Bit Planes	49
3.4 Spatial Resolution	52
3.5 Quantization and Dithering	54
3.6 Programs	63
Exercises	65
4 Point Processing	67
4.1 Introduction	67
4.2 Arithmetic Operations	68
4.3 Histograms	73
4.4 Lookup Tables	82
Exercises	83

5	Neighborhood Processing	85
5.1	Introduction	85
5.2	Notation	89
5.3	Filtering in MATLAB and Octave	91
5.4	Filtering in Python	94
5.5	Frequencies; Low and High Pass Filters	95
5.6	Gaussian Filters	100
5.7	Edge Sharpening	104
5.8	Non-Linear Filters	110
5.9	Edge-Preserving Blurring Filters	113
5.10	Region of Interest Processing	119
5.11	Programs	122
	Exercises	122
6	Image Geometry	125
6.1	Interpolation of Data	125
6.2	Image Interpolation	127
6.3	General Interpolation	130
6.4	Enlargement by Spatial Filtering	134
6.5	Scaling Smaller	137
6.6	Rotation	139
6.7	Correcting Image Distortion	143
	Exercises	146
7	The Fourier Transform	149
7.1	Introduction	149
7.2	Background	149
7.3	The One-Dimensional Discrete Fourier Transform	151
7.4	Properties of the One-Dimensional DFT	155
7.5	The Two-Dimensional DFT	161
7.6	Experimenting with Fourier Transforms	166
7.7	Fourier Transforms of Synthetic Images	169
7.8	Filtering in the Frequency Domain	173
7.9	Homomorphic Filtering	184
7.10	Programs	188
	Exercises	190
8	Image Restoration	193
8.1	Introduction	193
8.2	Noise	194
8.3	Cleaning Salt and Pepper Noise	197
8.4	Cleaning Gaussian Noise	202
8.5	Removal of Periodic Noise	207
8.6	Inverse	211
8.7	Wiener Filtering	218
	Exercises	220
9	Image Segmentation	223
9.1	Introduction	223
9.2	Thresholding	223
9.3	Applications of Thresholding	226

9.4	Choosing an Appropriate Threshold Value	227
9.5	Adaptive Thresholding	234
9.6	Edge Detection	237
9.7	Derivatives and Edges	238
9.8	Second Derivatives	244
9.9	The Canny Edge Detector	248
9.10	Corner Detection	251
9.11	The Hough and Radon Transforms	258
	Exercises	265
10	Mathematical Morphology	271
10.1	Introduction	271
10.2	Basic Ideas	271
10.3	Dilation and Erosion	272
10.4	Opening and Closing	282
10.5	The Hit-or-Miss Transform	287
10.6	Some Morphological Algorithms	289
10.7	A Note on the <code>bwmorph</code> Function in MATLAB and Octave	295
10.8	Grayscale Morphology	296
10.9	Applications of Grayscale Morphology	302
10.10	Programs	305
	Exercises	306
11	Image Topology	309
11.1	Introduction	309
11.2	Neighbors and Adjacency	310
11.3	Paths and Components	310
11.4	Equivalence Relations	311
11.5	Component Labeling	312
11.6	Lookup Tables	316
11.7	Distances and Metrics	318
11.8	Skeletonization	324
11.9	Programs	341
	Exercises	345
12	Shapes and Boundaries	351
12.1	Introduction	351
12.2	Chain Codes and Shape Numbers	351
12.3	Fourier Descriptors	362
	Exercises	369
13	Color Processing	371
13.1	What Is Color?	371
13.2	Color Models	375
13.3	Manipulating Color Images	379
13.4	Pseudocoloring	382
13.5	Processing of Color Images	388
13.6	Programs	398
	Exercises	400

14 Image Coding and Compression	403
14.1 Lossless and Lossy Compression	403
14.2 Huffman Coding	403
14.3 Run Length Encoding	406
14.4 Dictionary Coding: LZW Compression	410
14.5 The JPEG Algorithm	413
14.6 Programs	425
Exercises	429
15 Wavelets	431
15.1 Waves and Wavelets	431
15.2 A Simple Wavelet: The Haar Wavelet	436
15.3 Wavelets and Images	441
15.4 The Daubechies Wavelets	446
15.5 Image Compression Using Wavelets	450
15.6 High Pass Filtering Using Wavelets	455
15.7 Denoising Using Wavelets	456
Exercises	457
16 Special Effects	459
16.1 Polar Coordinates	459
16.2 Ripple Effects	463
16.3 General Distortion Effects	466
16.4 Pixel Effects	473
16.5 Color Images	475
Exercises	476
A Introduction to MATLAB and Octave	479
A.1 Introduction	479
A.2 Basic Use	480
A.3 Variables and the Workspace	482
A.4 Dealing with Matrices	484
A.5 Plots	496
A.6 Online Help	498
A.7 Programming	500
Exercises	502
B Introduction to Python	505
B.1 Basic Use	506
B.2 Arrays	509
B.3 Graphics and Plots	512
B.4 Programming	513
C The Fast Fourier Transform	517
Bibliography	525

Preface

Human beings are predominantly visual creatures, and our computing environments reflect this. We have the World Wide Web, filled with images of every possible type and provenance; and our own computers are crammed with images from the operating system, downloaded from elsewhere, or taken with our digital cameras. Then there are the vast applications which use digital images: remote sensing and satellite imaging; astronomy; medical imaging; microscopy; industrial inspection and so on.

This book is an introduction to *digital image processing*, from a strictly elementary perspective. We have selected only those topics that can be introduced with simple mathematics; however, these topics provide a very broad introduction to the discipline. The initial text was published in 2004; the current text is a rewritten version, with many changes.

This book is based on some very successful image processing subjects that have been taught at Victoria University in Melbourne, Australia, as well as in Singapore and Kuala Lumpur, Malaysia. The topics chosen, and their method of presentation, are the result of many hours talking to, and learning from, the hundreds of students who have taken these subjects.

There are a great many books on the subject of image processing, and this book differs from all of them in several important respects:

Not too much mathematics. Some mathematics is necessary for the explanation and discussion of image processing algorithms. But this book attempts to keep the mathematics at a level commensurate with elementary undergraduate computer science. The level of mathematics required is about one year's study at a tertiary level, including calculus and linear algebra.

A discrete approach. Since digital images are discrete entities, we have adopted an approach using mainly discrete mathematics. Although calculus is required for the development of some image processing topics, we attempt to connect the calculus-based (continuous) theory with its discrete implementation.

A strong connection between theory and practice. Since we have used a mainly discrete approach to explain the material, it becomes much easier to extend the theory into practice. This becomes particularly significant when we develop our own functions for implementing specific image processing algorithms.

Software based. There are image processing books that are based around programming languages: generally C or Java. The problem is that to use such software, a specialized image processing library must be used, and at least for C, there is no standard for this. And a problem with the Java image processing libraries is they are not really suitable for beginning students.

This book is based entirely around three different systems: MATLAB[®] and its Image Processing Toolbox, GNU Octave and its Image Processing Toolbox, and Python with various scientific and imaging libraries. Each system provides a complete environment for image processing that is easy to use, easy to explain, and easy to extend.

Plenty of examples. *All* the example images in this text are accompanied by commands in each of the three systems. Thus, if you work carefully through the book, you can create the images as given in the text.

Exercises. Most chapters finish off with a selection of exercises enabling the student to consolidate and extend the material. Some of the exercises are “pencil-and-paper”; designed for better understanding of the material; others are computer based to explore the algorithms and methods of that chapter.

What Is in the Book

The first three chapters set the scene for much of the rest of the book: exploring the nature and use of digital images, and how they can be obtained, stored, and displayed. Chapter 1 provides a brief introduction to the field of image processing, and attempts to give some idea as to its scope and areas of practice. We also define some common terms. Chapter 2 shows how images can be handled as matrices, and how the manipulation of these matrices forms the background of all subsequent work. Chapter 3 investigates aspects of image display, and looks at resolution and quantization and how these affect the appearance of the image. Appendix A provides a background in the use of MATLAB and Octave, and gives a brief introduction to programming in them.

Chapter 4 looks at some of the simplest, yet most powerful and widely used, of all image processing algorithms. These are the *point operations*, where the value of a pixel (a single “dot” in a digital image) is changed according to a single function of its value.

Chapter 5 introduces spatial filtering. Spatial filtering can be used for a vast range of image processing operations: from removing unnecessary detail, to sharpening edges and removing noise.

Chapter 6 looks at the geometry of an image: its size and orientation. Resizing an image may be necessary for inclusion in a web page or printed text; we may need to reduce it to fit or enlarge it.

Chapter 7 introduces the Fourier transform. This is possibly the single most important transform for image processing. To get a feeling for how the Fourier transform “works” and what information it provides, we need to spend some time exploring its mathematical foundations. This is the heaviest mathematics in this book, and requires some knowledge of complex numbers. In keeping with our philosophy, we use discrete mathematics only. We then show how images can be processed with great efficiency using the Fourier transform, and how various operations can be performed only using the Fourier transform.

Chapter 8 discusses the restoration of an image from different forms of degradation. Among these is the problem of *noise*, or “errors” in an image. Such errors are a natural consequence of electronic transmission of image signals, and although error correction of the signal can go a long way to ensuring that the image arrives “clean,” we may still receive images with noise. We also look at the removal of blur.

Chapter 9 addresses the problems of thresholding and of finding edges in an image. Edges are a vital aspect of object recognition: we can classify the size, shape, and type of object by an analysis of its edges. As well, edges form a vital aspect of human visual interpretation, and so the sharpening of edges is often an important part of image enhancement.

Chapter 10 introduces *morphology* or *mathematical morphology*, which is an area of image processing very much wedded to set theory. Historically, morphology developed from the need for granulometry, or the measurement of grains in ore samples. It is now a very powerful method for investigating shapes and sizes of objects. Morphology is generally

defined in terms of binary images (which is what we do here) and then can be extended to grayscale images. With the latter, we can also perform edge detection and some noise reduction.

Chapter 11 investigates the *topology* of digital images. This is concerned with the neighborhoods of pixels, and how the exploration of different neighborhoods leads to an understanding of the structure of image objects.

We continue the investigation of shapes in Chapter 12, but from a more spatial viewpoint; we look at traversing the edges of an object, and how the traversal can be turned into descriptors of the size and shape of the object.

Chapter 13 looks at color. Color is one of most important aspects of human interpretation. We look at the definition of color, from physical and digital perspectives, and how a color image can be processed using the techniques we have developed so far.

Chapter 14 discusses some basic aspects of image compression. Image files tend to be large, and their compression can be a matter of some concern, especially if there are many of them. We distinguish two types of compression: *lossless*, where there is no loss of information, and *lossy*, where higher compression rates can be obtained at the cost of losing some information.

Chapter 15 introduces wavelets. These have become a very hot topic in image processing; in some places they are replacing the use of the Fourier transform. Our treatment is introductory only; we show how wavelets and waves differ; how wavelets can be defined; how they can be applied to images; and the affects that can be obtained. In particular, we look at image compression, and show how wavelets can be used to obtain very high rates of lossy compression, with no apparent loss of image quality.

The final chapter is intended to be a bit more light-hearted than the others: here we look at some “special effects” on images. These are often provided with image editing programs—if you have a digital camera, chances are the accompanying software will allow this—our treatment attempts to provide an understanding into the nature of these algorithms.

What This Book Is Not

This book is *not* an introduction to MATLAB, Octave, Python, or their image processing tools. We have used only a small fraction of the many commands and functions available in each system—only those useful for an elementary text such as this. There is an enormous number of books on MATLAB available; fine general introductions are provided by the texts by Attaway [2] and Palm [20]. Python is equally well represented with texts; the texts by Langtangen [28], McKinney [30], and Howse [19] are all excellent basic references, even if Howse uses a different imaging library to the one in this text. Octave is less well represented by printed material, but the text by Quarteroni et al. [36] is very good. To really get to grips with any of these systems or their image processing tools and libraries, you can browse the excellent manuals online or through the system’s own help interface.

How to Use This Book

This book can be used for two separate streams of image processing; one very elementary, another a little more advanced. A first course consists of the following:

- Chapter 1
- Chapter 2 except for Section 2.5
- Chapter 3 except for Section 3.5

- Chapter 4
- Chapter 5
- Chapter 7
- Chapter 8
- Chapter 9 except for Sections 9.4, 9.5 and 9.9
- Chapter 10 except for Sections 10.8 and 10.9
- Chapter 13
- Chapter 14, Sections 14.2 and 14.3 only

A second course fills in the gaps:

- Section 2.5
- Section 3.5
- Chapter 6
- Sections 9.4, 9.5 and 9.9
- Sections 10.8 and 10.9
- Chapter 11
- Chapter 12
- Section 14.5
- Chapter 15

As taught at Victoria University, for the first course we concentrated on introducing students to the principles and practices of image processing, and did not worry about the implementation of the algorithms. In the second course, we spent much time discussing programming issues, and encouraged the students to write their own programs, and edit programs already written. Both courses have been very popular since their inception.

Changes Since the First Edition

The major change is that MATLAB, the sole computing environment in the first edition, has been joined by two other systems: GNU Octave [10], which in many ways is an open-source version of MATLAB, and Python [12], which is a deservedly popular general programming language. Python has various libraries for the handling of matrices and images, and is fast becoming recognized as a viable alternative to systems such as MATLAB.

At the time of writing the first edition, MATLAB was the the software of choice: its power, ease of use, and its functionality gave it few, if any, competitors. But the world of software has expanded greatly since then, and the beginning user is now spoiled for choice. In particular, many open-source systems (such as GNU Octave) have reached a maturity that makes them genuine alternatives to MATLAB for scientific programming. For users

who prefer the choice that open-source provides, this has been a welcome development. MATLAB still has an edge in its enormous breadth: its image processing toolbox has over 300 commands and functions—far more than any competitor—as well as a very mature graphics interface library. However, for basic and fundamental image processing—such as discussed in this text—MATLAB is equalled by both Octave and Python, and in some respects even surpassed by them.

Octave has been designed to be a “drop-in” replacement for MATLAB: standard MATLAB functions and scripts should run unchanged in Octave. This philosophy has been extended to the Octave Image Processing Toolbox, which although less comprehensive than that of MATLAB, is equal in depth. Thus, all through this text, except in a very few places, MATLAB and Octave are treated as though they are one and the same.

Python has several imaging libraries; I have chosen to use scikit-image [53]. This is one of several toolkits that are designed to extend and deepen the functionality in the standard Python scientific library SciPy (“Scikit” is a shortened version of “SciPy Toolkit”). It is designed to be easy to use: an image is just an array, instead of a specialized data-type unique to the toolkit. It is also implemented in Python, with the static library Cython used when fast performance is necessary. Other imaging libraries, such as Mahotas [9] and OpenCV [4, 19], while excellent, are implemented in C++. This makes it hard for the user (without a good knowledge of C++) to understand or extend the code. The combination of scikit-image with the functionality already available in SciPy provides very full-featured and robust image processing.

Many teachers, students, and practitioners are now attempting to find the best software for their needs. One of the aims of this text is to provide several different options, or alternatively enable users to migrate from one system to another.

There are also many changes throughout the text—some minor and typographical, some major. All the diagrams have been redrawn using the amazing TiKZ package [31]; some have been merely rewritten using TiKZ; others have been redrawn for greater symmetry, clarity, and synthesis of their graphics elements. Programs (in all three systems) have been written to be as modular as possible: in the first edition they tended to be monolithic and large. Modularity means greater flexibility, code reuse, and often conciseness—one example in the first edition which took most of a page, and 48 lines of small type, has been reduced to 9 lines, and includes greater functionality! Such large programs as are left have been relegated to the ends of the chapters.

All chapters have been written to be “system neutral”: as far as possible, imaging techniques are implemented in MATLAB and Octave, and again in Python. In Chapter 2, the section on PNG images has been expanded to provide more information about this format. Chapter 5 has been extended with some material about the Kuwahara and bilateral filters—both methods for blurring an image while maintaining edge sharpness—as well as a rewritten section on region of interest processing. In Chapter 6 an original section on anamorphosis has been removed, on account of difficulties obtaining a license to use a particular image; this section has been replaced with a new section containing an example of image warping. For readers who may pine for anamorphosis, the excellent recent text by Steven Tanimoto [50] contains much wonderful material on anamorphosis, as well as being written in a beautifully approachable style. Chapter 7 has had a few new diagrams inserted, hopefully to clarify some of the material. Chapter 9 includes an extended discussion on Otsu’s method of thresholding, as well as new material about the ISODATA method. This chapter also contains a new section about corner detection, with discussions of both the Moravec and Harris-Stephens operators. The section on the Hough transform has been completely rewritten to include the Radon transform. In Chapter 11, the algorithms for the Zhang-Suen and Guo-Hall skeletonization methods have been rewritten to be simpler. Chapter 12 contains a new discussion on Fourier shape descriptors, with different examples.

Chapter 13 contains a new section at the end introducing one version of the retinex algorithm. Chapter 14 implements a simpler and faster method for run-length encoding, and includes a new section on LZW compression. Chapter 15 has been simplified, and more connections made between the discrete wavelet transform and digital filtering. Also in this chapter a new wavelet toolbox is used, one that amazingly has been written to be used in each of MATLAB, Octave, and Python. In Chapter 16 the greater modularity of the programs has allowed a greater conciseness, with (I hope) increased clarity.

Finally, new images have been used to ameliorate licensing or copyright difficulties. All images in this book are either used with permission of their owners, or are the author's and may be used freely and without permission. They are listed at the end.

Acknowledgments

The first edition of this book began as set of notes written to accompany an undergraduate introductory course to image processing; these notes were then extended for a further course. Thanks must go to the many students who have taken these courses, and who have discussed the subject material and its presentation with me.

I would like to thank my colleagues at Victoria University for many stimulating conversations about teaching and learning. I would also like to thank Associate Professor Robyn Pierce at the University of Melbourne, for providing me with a very pleasant environment for six months, during which much of the initial programming and drafting of this new edition was done.

Sunil Nair and Sarfraz Khan, of Taylor and Francis Publishing Company, have provided invaluable help and expert advice, as well as answering all my emails with admirable promptness. Their helping hands when needed have made this redrafting and rewriting a very pleasant and enjoyable task.

I am indebted to the constant hard work of David Miguel Susano Pinto (maintainer of the Octave Forge image package under the alias of Carnë Draug), and his predecessor Søren Hauberg, the original author of the image package; also Stefan van der Walt, lead developer of the Python scikit-image package. They have made lasting and valuable contributions to open-source imaging software, and also have answered many of my questions.

I would also like to thank the reviewers for their close and careful reading of the initial proposal, and for their thoughtful, detailed, and constructive comments.

Finally, heartfelt thanks to my long suffering wife Felicity, and my children Angus, Edward, Fenella, William, and Finlay, who have put up, for far too long, with absent-mindedness, tables and benches covered with scraps of papers, laptops, and books, and a husband and father who was more concentrated on his writing than their needs.

A Note on the Images

Some of the images used in this book are the author's and may be used freely and without restriction. They are:

arch.png
backyard.png
blocks.png
bot_gardens.png
buffalo.png
car.png
cat.png
chickens.png
circles.png
circles2.png
color_sunset.png
emu.png
engineer.png
handmade.png
iris.png
koala.png
meet_text.png
monkey.png
morph_text.png
newborn.png
nicework.png
paperclips.png
pelicans.png
pinenuts.png
rings.png
seagull.png
stairs.png
street.png
sunset.png
twins.png
venice.png
wombats.png

MIT Press has kindly allowed the use of the standard test image

cameraman.png

The following images have been provided courtesy of [shutterstock.com](https://www.shutterstock.com):

blood.png
paramecium.png

The image

caribou.tif

has been cropped from the NOAA image [anim0614.jpg](#), a photograph of a caribou (*Rangifer tarandus*) by Capt. Budd Christman of the NOAA Corps.

The image

monastery.png

has been taken from <http://www.public-domain-image.com>; more particularly from <http://bit.ly/18V2Z57>; the photographer is Andrew McMillan.

The image

daisies.png

has been taken from www.pixabay.com as an image in the public domain; the photographer is Marianne Langenbach.

The iconic image

thylacine.png

showing the last known living thylacine, or Tasmanian tiger, at the Hobart zoo in 1933, is now in the public domain.

The x-ray image

xray.png

has been taken from Wikimedia Commons, as an image in the public domain. It can be found at <http://bit.ly/1Az0Tzk>; the original image has been provided by Diego Grez from Santa Cruz, Chile.

MATLAB and Simulink are registered trademarks of The Mathworks, Inc. For product information, please contact:

The Mathworks, Inc.
3 Apple Hill Drive
Natick, MA 01760-2098 USA
Tel: 508-647-7000
Fax: 508-647-7001
Email: info@mathworks.com
Web: www.mathworks.com

Chapter 1

Introduction

1.1 Images and Pictures

As we mentioned in the Preface, human beings are predominantly visual creatures: we rely heavily on our vision to make sense of the world around us. We not only look at things to identify and classify them, but we can scan for differences, and obtain an overall rough “feeling” for a scene with a quick glance.

Humans have evolved very precise visual skills: we can identify a face in an instant; we can differentiate colors; we can process a large amount of visual information very quickly.

However, the world is in constant motion: stare at something for long enough and it will change in some way. Even a large solid structure, like a building or a mountain, will change its appearance depending on the time of day (day or night); amount of sunlight (clear or cloudy), or various shadows falling upon it.

We are concerned with single images: snapshots, if you like, of a visual scene. Although image processing can deal with changing scenes, we shall not discuss it in any detail in this text.

For our purposes, an *image* is a single picture that represents something. It may be a picture of a person, people, or animals, an outdoor scene, a microphotograph of an electronic component, or the result of medical imaging. Even if the picture is not immediately recognizable, it will not be just a random blur.

1.2 What Is Image Processing?

Image processing involves changing the nature of an image in order to either

1. Improve its pictorial information for human interpretation
2. Render it more suitable for autonomous machine perception

We shall be concerned with *digital image processing*, which involves using a computer to change the nature of a *digital image* (see Section 1.4). It is necessary to realize that these two aspects represent two separate but equally important aspects of image processing. A procedure that satisfies condition (1)—a procedure that makes an image “look better”—may be the very worst procedure for satisfying condition (2). Humans like their images to be sharp, clear, and detailed; machines prefer their images to be simple and uncluttered.

Examples of (1) may include:

- Enhancing the edges of an image to make it appear sharper; an example is shown in Figure 1.1. Note how the second image appears “cleaner”; it is a more pleasant image. Sharpening edges is a vital component of printing: in order for an image to appear “at its best” on the printed page; some sharpening is usually performed.



(a) The original image



(b) Result after “sharpening”

FIGURE 1.1: Image sharpening

- Removing “noise” from an image; noise being random errors in the image. An example is given in Figure 1.2. Noise is a very common problem in data transmission: all sorts of electronic components may affect data passing through them, and the results may be undesirable. As we shall see in Chapter 8, noise may take many different forms, and each type of noise requires a different method of removal.



(a) The original image



(b) After removing noise

FIGURE 1.2: Removing noise from an image

- Removing motion blur from an image. An example is given in Figure 1.3. Note that in the deblurred image (b) it is easier to read the numberplate, and to see the spikes on the fence behind the car, as well as other details not at all clear in the original image (a). Motion blur may occur when the shutter speed of the camera is too long for the speed of the object. In photographs of fast moving objects—athletes, vehicles for example—the problem of blur may be considerable.



(a) The original image



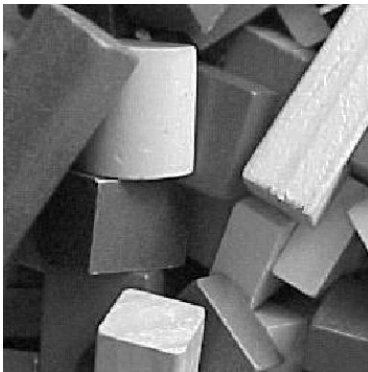
(b) After removing the blur

FIGURE 1.3: Image deblurring

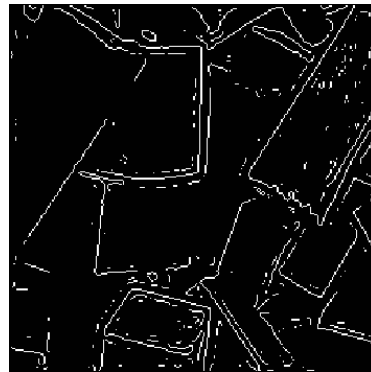
Examples of (2) may include:

- Obtaining the edges of an image. This may be necessary for the measurement of objects in an image; an example is shown in Figures 1.4. Once we have the edges we can measure their spread, and the area contained within them. We can also use edge detection algorithms as a first step in edge enhancement, as we saw previously.

From the edge result, we see that it may be necessary to enhance the original image slightly, to make the edges clearer.



(a) The original image

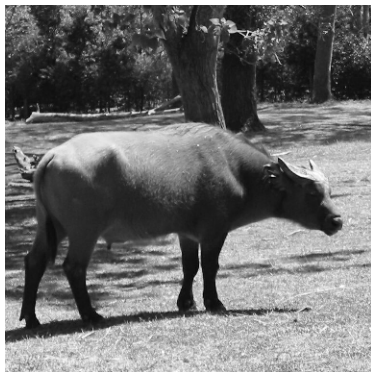


(b) Its edge image

FIGURE 1.4: Finding edges in an image

- Removing detail from an image. For measurement or counting purposes, we may not be interested in all the detail in an image. For example, a machine inspected items on an assembly line, the only matters of interest may be shape, size, or color. For such

cases, we might want to simplify the image. Figure 1.5 shows an example: in image (a) is a picture of an African buffalo, and image (b) shows a blurred version in which extraneous detail (like the logs of wood in the background) have been removed. Notice that in image (b) all the fine detail is gone; what remains is the coarse structure of the image. We could for example, measure the size and shape of the animal without being “distracted” by unnecessary detail.



(a) The original image



(b) Blurring to remove detail

FIGURE 1.5: Blurring an image

1.3 Image Acquisition and Sampling

Sampling refers to the process of digitizing a continuous function. For example, suppose we take the function

$$y = \sin(x) + \frac{1}{3} \sin(3x)$$

and sample it at ten evenly spaced values of x only. The resulting sample points are shown in Figure 1.6. This shows an example of *undersampling*, where the number of points is not sufficient to reconstruct the function. Suppose we sample the function at 100 points, as shown in Figure 1.7. We can clearly now reconstruct the function; all its properties can be determined from this sampling. In order to ensure that we have enough sample points, we require that the sampling period is not greater than one-half the finest detail in our function. This is known as the *Nyquist criterion*, and can be formulated more precisely in terms of “frequencies,” which are discussed in Chapter 7. The Nyquist criterion can be stated as the *sampling theorem*, which says, in effect, that a continuous function can be reconstructed from its samples provided that the sampling frequency is at least twice the maximum frequency in the function. A formal account of this theorem is provided by Castleman [7].

Sampling an image again requires that we consider the Nyquist criterion, when we consider an image as a continuous function of two variables, and we wish to sample it to produce a digital image.

An example is shown in Figure 1.8 where an image is shown, and then with an under-sampled version. The jagged edges in the undersampled image are examples of *aliasing*.



FIGURE 1.6: Sampling a function—undersampling

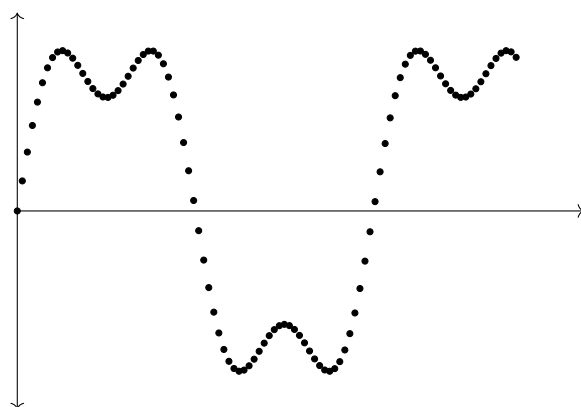
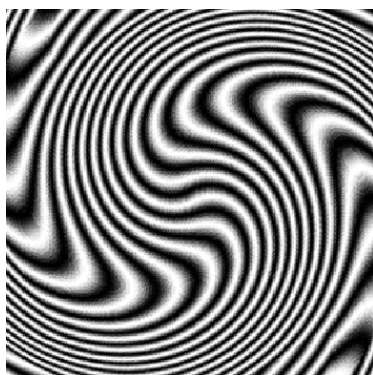
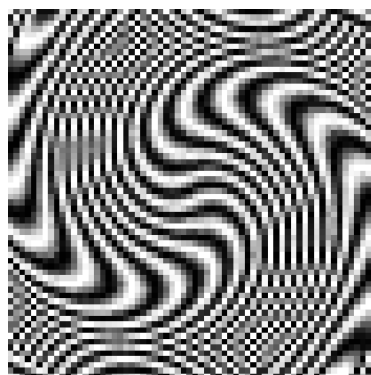


FIGURE 1.7: Sampling a function with more points



Correct sampling; no aliasing



An undersampled version with aliasing

FIGURE 1.8: Effects of sampling

The sampling rate will of course affect the final resolution of the image; we discuss this in Chapter 3. In order to obtain a sampled (digital) image, we may start with a continuous representation of a scene. To view the scene, we record the energy reflected from it; we may use visible light, or some other energy source.

Using Light

Light is the predominant energy source for images; simply because it is the energy source that human beings can observe directly. We are all familiar with photographs, which are a pictorial record of a visual scene.

Many digital images are captured using visible light as the energy source; this has the advantage of being safe, cheap, easily detected, and readily processed with suitable hardware. Two very popular methods of producing a digital image are with a digital camera or a flat-bed scanner.

CCD camera. Such a camera has, in place of the usual film, an array of *photosites*; these are silicon electronic devices whose voltage output is proportional to the intensity of light falling on them.

For a camera attached to a computer, information from the photosites is then output to a suitable storage medium. Generally this is done on hardware, as being much faster and more efficient than software, using a *frame-grabbing card*. This allows a large number of images to be captured in a very short time—in the order of one ten-thousandth of a second each. The images can then be copied onto a permanent storage device at some later time.

This is shown schematically in Figure 1.9.

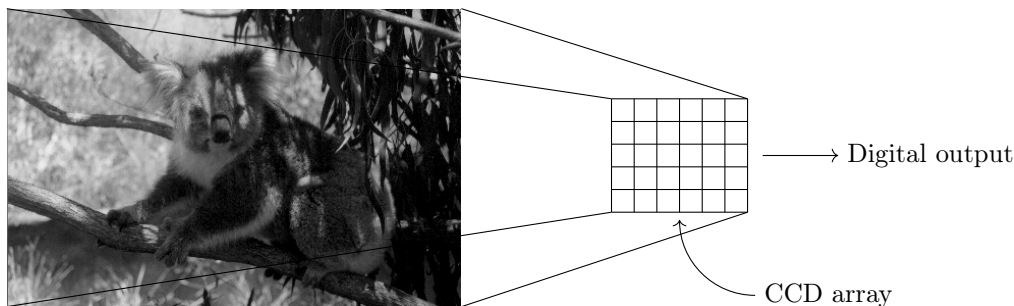


FIGURE 1.9: Capturing an image with a CCD array

The output will be an array of values; each representing a sampled point from the original scene. The elements of this array are called *picture elements*, or more simply *pixels*.

Digital still cameras use a range of devices, from floppy discs and CDs, to various specialized cards and “memory sticks.” The information can then be downloaded from these devices to a computer hard disk.

Flat bed scanner. This works on a principle similar to the CCD camera. Instead of the entire image being captured at once on a large array, a single row of photosites is moved across the image, capturing it row-by-row as it moves. This is shown schematically in Figure 1.10.

Since this is a much slower process than taking a picture with a camera, it is quite reasonable to allow all capture and storage to be processed by suitable software.



FIGURE 1.10: Capturing an image with a CCD scanner

Other Energy Sources

Although light is popular and easy to use, other energy sources may be used to create a digital image. Visible light is part of the *electromagnetic spectrum*: radiation in which the energy takes the form of waves of varying wavelength. These range from cosmic rays of very short wavelength, to electric power, which has very long wavelength. Figure 1.11 illustrates this. For microscopy, we may use x-rays or electron beams. As we can see from Figure 1.11,

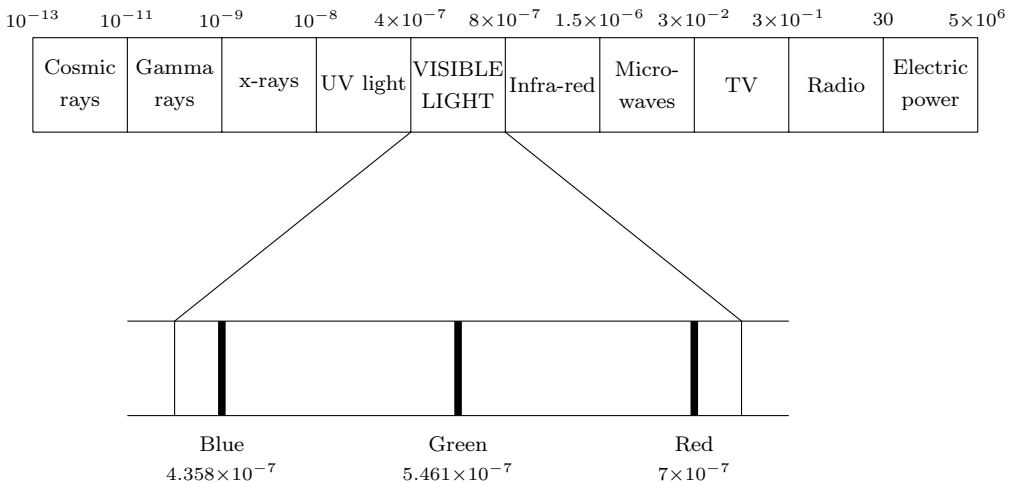


FIGURE 1.11: The electromagnetic spectrum

x-rays have a shorter wavelength than visible light, and so can be used to resolve smaller objects than are possible with visible light. See Clark [8] for a good introduction to this. X-rays are of course also useful in determining the structure of objects usually hidden from view, such as bones.

A further method of obtaining images is by the use of *x-ray tomography*, where an object is encircled by an x-ray beam. As the beam is fired through the object, it is detected on the other side of the object, as shown in Figure 1.12. As the beam moves around the object, an image of the object can be constructed; such an image is called a *tomogram*. In a CAT (Computed Axial Tomography) scan, the patient lies within a tube around which x-ray

beams are fired. This enables a large number of tomographic “slices” to be formed, which can then be joined to produce a three-dimensional image. A good account of such systems (and others) is given by Siedband [48].

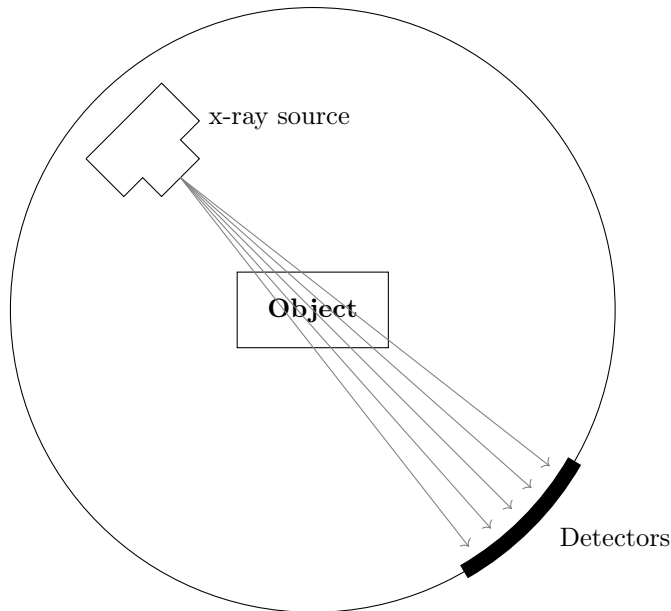


FIGURE 1.12: X-ray tomography

1.4 Images and Digital Images

Suppose we take an image, a photo, say. For the moment, let’s make things easy and suppose the photo is monochromatic (that is, shades of gray only), so no color. We may consider this image as being a two-dimensional function, where the function values give the brightness of the image at any given point, as shown in Figure 1.13. We may assume that in such an image brightness values can be any real numbers in the range 0.0 (black) to 1.0 (white). The ranges of x and y will clearly depend on the image, but they can take all real values between their minima and maxima.

Such a function can of course be plotted, as shown in Figure 1.14. However, such a plot is of limited use to us in terms of image analysis. The concept of an image as a function, however, will be vital for the development and implementation of image processing techniques.

A *digital image* differs from a photo in that the x , y , and $f(x, y)$ values are all *discrete*. Usually they take on only integer values, so the image shown in Figure 1.13 will have x and y ranging from 1 to 256 each, and the brightness values also ranging from 0 (black) to 255 (white). A digital image, as we have seen above, can be considered a large array of sampled points from the continuous image, each of which has a particular quantized brightness; these points are the *pixels*, which constitute the digital image. The pixels surrounding a given pixel constitute its *neighborhood*. A neighborhood can be characterized by its shape in the same way as a matrix: we can speak, for example, of a 3×3 neighborhood or of a 5×7

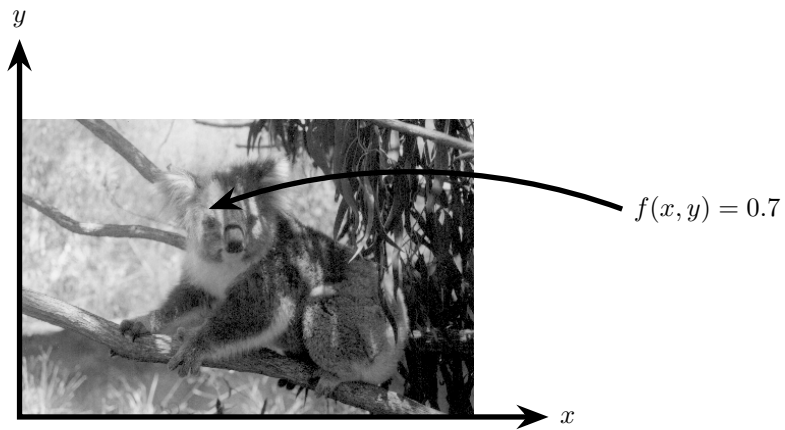


FIGURE 1.13: An image as a function

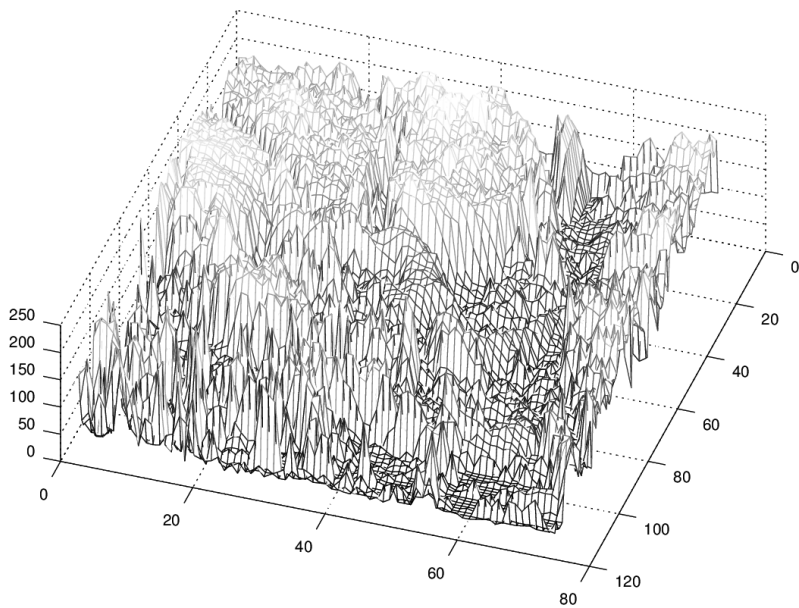


FIGURE 1.14: The image of Figure 1.13 plotted as a function of two variables

neighborhood. Except in very special circumstances, neighborhoods have odd numbers of rows and columns; this ensures that the current pixel is in the center of the neighborhood. An example of a neighborhood is given in Figure 1.15. If a neighborhood has an even number of rows or columns (or both), it may be necessary to specify which pixel in the neighborhood is the “current pixel.”

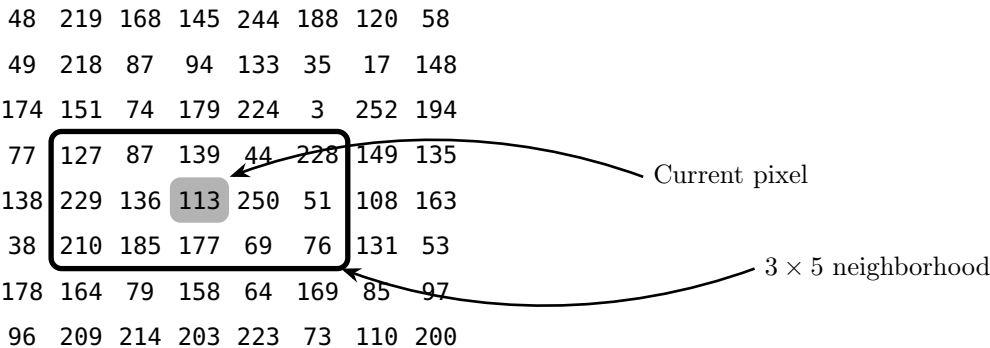


FIGURE 1.15: Pixels, with a neighborhood

1.5 Some Applications

Image processing has an enormous range of applications; almost every area of science and technology can make use of image processing methods. Here is a short list just to give some indication of the range of image processing applications.

1. Medicine
 - Inspection and interpretation of images obtained from x-rays, MRI, or CAT scans
 - Analysis of cell images, of chromosome karyotypes
2. Agriculture
 - Satellite/aerial views of land, for example to determine how much land is being used for different purposes, or to investigate the suitability of different regions for different crops
 - Inspection of fruit and vegetables—distinguishing good and fresh produce from old
3. Industry
 - Automatic inspection of items on a production line
 - Inspection of paper samples
4. Law enforcement
 - Fingerprint analysis
 - Sharpening or deblurring of speed-camera images

1.6 Image Processing Operations

It is convenient to subdivide different image processing algorithms into broad subclasses. There are different algorithms for different tasks and problems, and often we would like to distinguish the nature of the task at hand.

Image enhancement. This refers to processing an image so that the result is more suitable for a particular application. Examples include:

- Sharpening or deblurring an out of focus image
- Highlighting edges
- Improving image contrast, or brightening an image
- Removing noise

Image restoration. This may be considered as reversing the damage done to an image by a known cause, for example:

- Removing of blur caused by linear motion
- Removal of optical distortions
- Removing periodic interference

Image segmentation. This involves subdividing an image into constituent parts, or isolating certain aspects of an image:

- Finding lines, circles, or particular shapes in an image
- In an aerial photograph, identifying cars, trees, buildings, or roads

Image registration. This involves “matching” distinct images so that they can be compared, or processed together. The initial images must all be joined to share the same coordinate system. In this text, registration as such is not covered, but some tasks that are vital to registration are discussed, for example corner detection.

These classes are not disjoint; a given algorithm may be used for both image enhancement or for image restoration. However, we should be able to decide what it is that we are trying to do with our image: simply make it look better (enhancement) or removing damage (restoration).

1.7 An Image Processing Task

We will look in some detail at a particular real-world task, and see how the above classes may be used to describe the various stages in performing this task. The job is to obtain, by an automatic process, the postcodes from envelopes. Here is how this may be accomplished:

Acquiring the image. First we need to produce a digital image from a paper envelope. This can be done using either a CCD camera or a scanner.

Preprocessing. This is the step taken before the “major” image processing task. The problem here is to perform some basic tasks in order to render the resulting image more suitable for the job to follow. In this case, it may involve enhancing the contrast, removing noise, or identifying regions likely to contain the postcode.

Segmentation. Here is where we actually “get” the postcode; in other words, we extract from the image that part of it that contains just the postcode.

Representation and description. These terms refer to extracting the particular features which allow us to differentiate between objects. Here we will be looking for curves, holes, and corners, which allow us to distinguish the different digits that constitute a postcode.

Recognition and interpretation. This means assigning labels to objects based on their descriptors (from the previous step), and assigning meanings to those labels. So we identify particular digits, and we interpret a string of four digits at the end of the address as the postcode.

1.8 Types of Digital Images

We shall consider four basic types of images:

Binary. Each pixel is just black or white. Since there are only two possible values for each pixel, we only need one bit per pixel. Such images can therefore be very efficient in terms of storage. Images for which a binary representation may be suitable include text (printed or handwriting), fingerprints, or architectural plans.

An example was the image shown in Figure 1.4(b). In this image, we have only the two colors: white for the edges, and black for the background. See Figure 1.16.

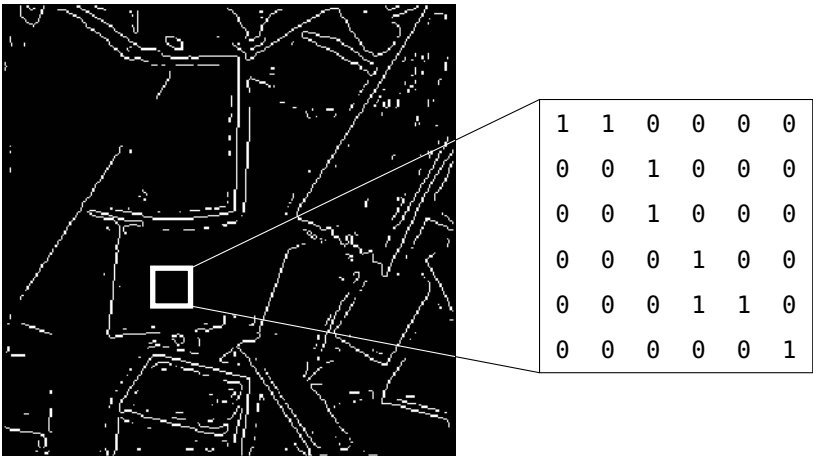


FIGURE 1.16: A binary image

Grayscale. Each pixel is a shade of gray, normally from 0 (black) to 255 (white). This range means that each pixel can be represented by eight bits, or exactly one byte. This is a very natural range for image file handling. Other grayscale ranges are used,

but generally they are a power of 2. Such images arise in medicine (X-rays), images of printed works, and indeed 256 different gray levels is sufficient for the recognition of most natural objects.

An example is the street scene shown in Figure 1.1, and in Figure 1.17.

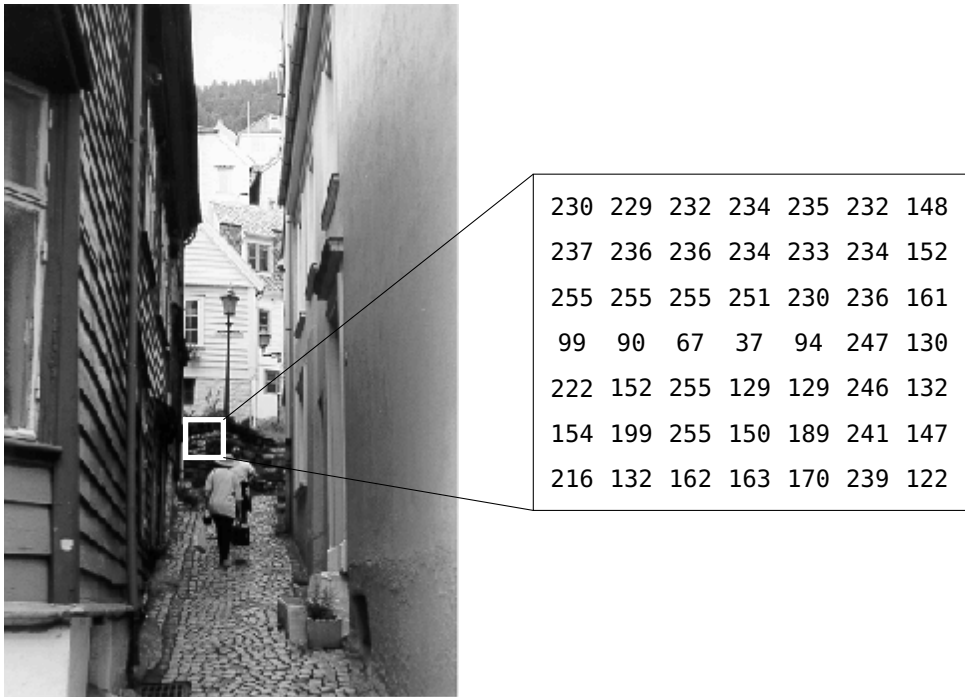


FIGURE 1.17: A grayscale image

True color, or RGB. Here each pixel has a particular color; that color being described by the amount of red, green, and blue in it. If each of these components has a range 0–255, this gives a total of $255^3 = 16,777,216$ different possible colors in the image. This is enough colors for any image. Since the total number of bits required for each pixel is 24, such images are also called *24-bit color images*.

Such an image may be considered as consisting of a “stack” of three matrices; representing the red, green, and blue values for each pixel. This means that for every pixel there are three corresponding values.

An example is shown in Figure 1.18.

Indexed. Most color images only have a small subset of the more than sixteen million possible colors. For convenience of storage and file handling, the image has an associated *color map*, or *color palette*, which is simply a list of all the colors used in that image. Each pixel has a value which does not give its color (as for an RGB image), but an *index* to the color in the map.

It is convenient if an image has 256 colors or less, for then the index values will only require one byte each to store. Some image file formats (for example, Compuserve GIF) allow only 256 colors or fewer in each image, for precisely this reason.

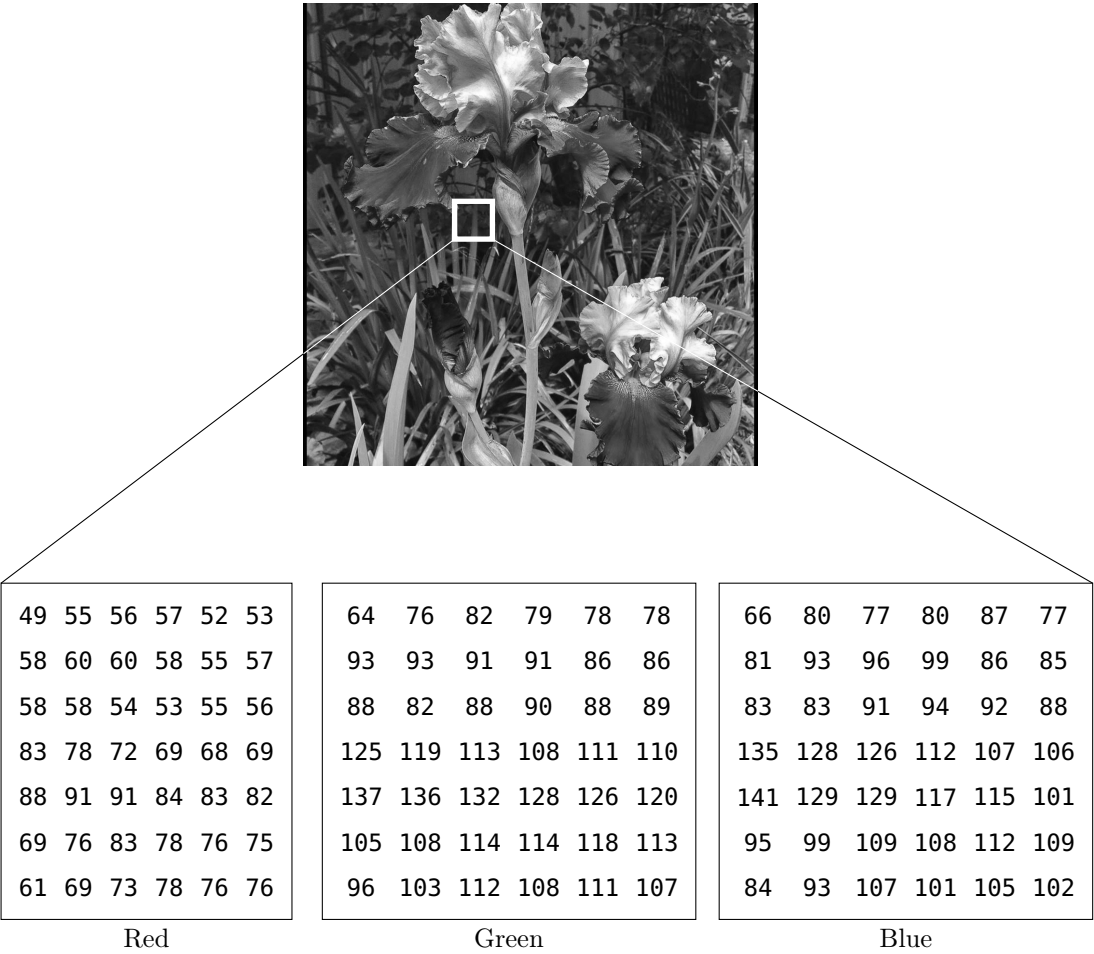


FIGURE 1.18: **SEE COLOR INSERT** A true color image

Figure 1.19 shows an example. In this image the indices, rather than being the gray values of the pixels, are simply indices into the color map. Without the color map, the image would be very dark and colorless. In the figure, for example, pixels labelled 5 correspond to 0.2627 0.2588 0.2549, which is a dark grayish color.

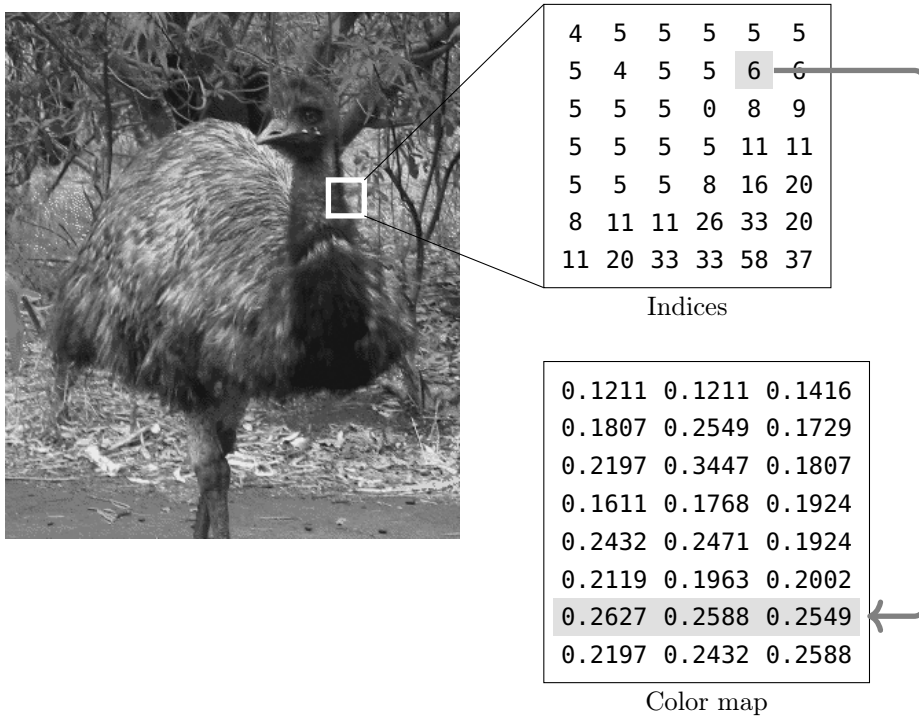


FIGURE 1.19: **SEE COLOR INSERT** An indexed color image

1.9 Image File Sizes

Image files tend to be large. We shall investigate the amount of information used in different image types of varying sizes. For example, suppose we consider a 512×512 binary image. The number of bits used in this image (assuming no compression, and neglecting, for the sake of discussion, any header information) is

$$\begin{aligned}
 512 \times 512 \times 1 &= 262,144 \\
 &= 32,768 \text{ bytes} \\
 &= 32.768 \text{ Kb} \\
 &\approx 0.033 \text{ Mb.}
 \end{aligned}$$

(Here we use the convention that a kilobyte is 1000 bytes, and a megabyte is one million bytes.)

A grayscale image of the same size requires:

$$\begin{aligned} 512 \times 512 \times 1 &= 262,144 \text{ bytes} \\ &= 262.14 \text{ Kb} \\ &\approx 0.262 \text{ Mb.} \end{aligned}$$

If we now turn our attention to color images, each pixel is associated with 3 bytes of color information. A 512×512 image thus requires

$$\begin{aligned} 512 \times 512 \times 3 &= 786,432 \text{ bytes} \\ &= 786.43 \text{ Kb} \\ &\approx 0.786 \text{ Mb.} \end{aligned}$$

Many images are of course larger than this; satellite images may be of the order of several thousand pixels in each direction.

1.10 Image Perception

Much of image processing is concerned with making an image appear “better” to human beings. We should therefore be aware of the limitations of the human visual system. Image perception consists of two basic steps:

1. Capturing the image with the eye
2. Recognizing and interpreting the image with the *visual cortex* in the brain

The combination and immense variability of these steps influences the ways in which we perceive the world around us.

There are a number of things to bear in mind:

1. *Observed intensities* vary as to the background. A single block of gray will appear darker if placed on a white background than if it were placed on a black background. That is, we don’t perceive gray scales “as they are,” but rather as they differ from their surroundings. In Figure 1.20, a gray square is shown on two different backgrounds. Notice how much darker the square appears when it is surrounded by a light gray. However, the two central squares have exactly the same intensity.
2. We may observe non-existent intensities as bars in continuously varying gray levels. See, for example, Figure 1.21. This image varies continuously from light to dark as we travel from left to right. However, it is impossible for our eyes not to see a few horizontal edges in this image.
3. Our visual system tends to undershoot or overshoot around the boundary of regions of different intensities. For example, suppose we had a light gray blob on a dark gray background. As our eye travels from the dark background to the light region, the boundary of the region appears lighter than the rest of it. Conversely, going in the other direction, the boundary of the background appears *darker* than the rest of it.



FIGURE 1.20: A gray square on different backgrounds



FIGURE 1.21: Continuously varying intensities

Exercises

1. Watch the TV news, and see if you can observe any examples of image processing.
2. If your TV set allows it, turn down the color as far as you can to produce a monochromatic display. How does this affect your viewing? Is there anything that is hard to recognize without color?
3. Look through a collection of old photographs. How can they be enhanced or restored?
4. For each of the following, list five ways in which image processing could be used:
 - Medicine
 - Astronomy
 - Sport
 - Music
 - Agriculture
 - Travel
5. Image processing techniques have become a vital part of the modern movie production process. Next time you watch a film, take note of all the image processing involved.
6. If you have access to a scanner, scan in a photograph, and experiment with all the possible scanner settings.
 - (a) What is the smallest sized file you can create which shows all the detail of your photograph?
 - (b) What is the smallest sized file you can create in which the major parts of your image are still recognizable?
 - (c) How do the color settings affect the output?
7. If you have access to a digital camera, again photograph a fixed scene, using all possible camera settings.
 - (a) What is the smallest file you can create?
 - (b) How do the light settings affect the output?
8. Suppose you were to scan in a monochromatic photograph, and then print out the result. Then suppose you scanned in the printout, and printed out the result of that, and repeated this a few times. Would you expect any degradation of the image during this process? What aspects of the scanner and printer would minimize degradation?
9. Look up *ultrasonography*. How does it differ from the image acquisition methods discussed in this chapter? What is it used for? If you can, compare an ultrasound image with an x-ray image. How do they differ? In what ways are they similar?
10. If you have access to an image viewing program (other than MATLAB, Octave or Python) on your computer, make a list of the image processing capabilities it offers. Can you find imaging tasks it is unable to do?

Bibliography

- [1] Adobe Developers Association. *TIFF Revision 6.0*. available at <http://partners.adobe.com/public/developer/en/tiff/TIFF6.pdf>. Adobe Systems, Inc., 1992.
- [2] Stormy Attaway. *Matlab: A Practical Introduction to Programming and Problem Solving*. 2nd Edition. Butterworth-Heinemann, 2013.
- [3] Gregory A. Baxes. *Digital Image Processing: Principles and Applications*. John Wiley & Sons, 1994.
- [4] Gary Bradski and Adrian Kaehler. *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, Inc., 2008.
- [5] Wayne C. Brown and Barry J. Shepherd. *Graphics File Formats: Reference and Guide*. Manning Publications, 1995.
- [6] John Canny. "A computational approach to edge detection". In: *IEEE Transactions in Pattern Analysis and Machine Intelligence* 6 (1986), pp. 679–698.
- [7] Kenneth R. Castleman. *Digital Image Processing*. Prentice Hall, 1996.
- [8] Ashley R. Clark and Colin N Eberhardt. *Microscopy Techniques for Materials Science*. CRC Press, Boca Raton, FL, 2002.
- [9] Luis Pedro Coelho. "Mahotas: Open source software for scriptable computer vision". In: *arXiv preprint arXiv:1211.4907* (2012).
- [10] Octave community. *GNU Octave 3.8.1*. 2014. URL: `\url{www.gnu.org/software/octave/}`.
- [11] James D. Foley et al. *Introduction to Computer Graphics*. Addison-Wesley, 1994.
- [12] Python Software Foundation. *Python Language Reference, v2.7*. available at <http://www.python.org>.
- [13] Rafael Gonzalez and Richard E. Woods. *Digital Image Processing*. Third Edition. Addison-Wesley, 2007.
- [14] Robert M. Haralick and Linda G. Shapiro. *Computer and Robot Vision*. Addison-Wesley, 1993.
- [15] Stephen Hawley. "Ordered dithering". In: *Graphics Gems*. Ed. by Andrew S. Glassner. Academic Press, 1990, pp. 176–178.
- [16] M. D. Heath et al. "A robust visual method for assessing the relative performance of edge-detection algorithms". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 19.2 (1997), pp. 1338–1359.
- [17] Robert V. Hogg and Allen T. Craig. *Introduction to Mathematical Statistics*. Fifth. Prentice-Hall, 1994.
- [18] Gerard J. Holzmann. *Beyond Photography: the Digital Darkroom*. Prentice Hall, 1988.
- [19] Joseph Howse. *OpenCV Computer Vision with Python*. Packt Publishing Ltd, 2013.
- [20] William J. Palm III. *Introduction to MATLAB for Engineers*. Third edition. McGraw-Hill, 2010.
- [21] Adobe Systems Incorporated. *PostScript(R) Language Reference*. Third Edition. Addison-Wesley Publishing Co., 1999.
- [22] Anil K. Jain. *Fundamentals of Digital Image Processing*. Prentice Hall, 1989.
- [23] Glyn James and David Burley. *Advanced Modern Engineering Mathematics*. Second Edition. Addison-Wesley, 1999.

- [24] Arne Jensen and Anders la Cour-Harbo. *Ripples in Mathematics: the Discrete Wavelet Transform*. Springer Science & Business Media, 2001.
- [25] Daniel J Jobson, Z-U Rahman, and Glenn A Woodell. "Properties and performance of a center/surround retinex". In: *Image Processing, IEEE Transactions on* 6.3 (1997), pp. 451–462.
- [26] David C. Kay and John R. Levine. *Graphics File Formats*. Windcrest/McGraw-Hill, 1995.
- [27] Edwin H. Land. "The Retinex Theory of Color Vision". In: *Scientific American* 237.6 (1977), pp. 108–128.
- [28] Hans Petter Langtangen. *A Primer on Scientific Programming with Python*. Springer, 2011.
- [29] Jae S. Lim. *Two-Dimensional Signal and Image Processing*. Prentice Hall, 1990.
- [30] Wes McKinney. *Python for data analysis: Data wrangling with Pandas, NumPy, and IPython*. "O'Reilly Media, Inc.", 2012.
- [31] Andrew Mertz and William Slough. "Graphics with TikZ". In: *The PracTEX Journal* 1 (2007).
- [32] James R. Parker. *Algorithms for Image Processing and Computer Vision*. John Wiley and Sons, 1997.
- [33] Jim R Parker. *Algorithms for Image Processing and Computer Vision*. John Wiley & Sons, 2010.
- [34] Maria Petrou and Pangioti Bosdogianni. *Image Processing: the Fundamentals*. John Wiley and Sons, 1999.
- [35] William K Pratt. *Introduction to Digital Image Processing*. CRC Press, 2013.
- [36] Alfio Quarteroni, Fausto Saleri, and Paola Gervasio. *Scientific Computing with MATLAB and Octave*. Springer, 2006.
- [37] Majid Rabbani and Paul W. Jones. *Digital Image Compression Techniques*. SPIE Optical Engineering Press, 1991.
- [38] Greg Roelofs. *PNG: The Definitive Guide*. O'Reilly and Associates, 1999.
- [39] Steven Roman. *Introduction to Coding and Information Theory*. Springer-Verlag, 1997.
- [40] Azriel Rosenfeld and Avinash C. Kak. *Digital Picture Processing*. Second Edition. Academic Press, 1982.
- [41] David Salomon. *A Concise Introduction to Data Compression*. Springer Science & Business Media, 2007.
- [42] David Salomon and Giovanni Motta. *Handbook of Data Compression*. Springer Science & Business Media, 2010.
- [43] Khalid Sayood. *Introduction to Data Compression*. Newnes, 2012.
- [44] Dale A. Schumacher. "A comparison of digital halftoning techniques". In: *Graphics Gems II*. Ed. by James Arvo. Academic Press, 1991, pp. 57–71.
- [45] Jean Paul Serra. *Image Analysis and Mathematical Morphology*. Academic Press, 1982.
- [46] Frank Y Shih. *Image Processing and Mathematical Morphology: Fundamentals and Applications*. CRC press, 2009.
- [47] Frank Y Shih. *Image Processing and Pattern Recognition: Fundamentals and Techniques*. John Wiley & Sons, 2010.
- [48] Melvin P. Siedband. "Medical Imaging Systems". In: *Medical instrumentation : application and design*. Ed. by John G. Webster. John Wiley and Sons, 1998, pp. 518–576.
- [49] Milan Sonka, Vaclav Hlavac, and Roger Boyle. *Image Processing, Analysis and Machine Vision*. Second Edition. PWS Publishing, 1999.
- [50] Steven Tanimoto. *Structured Computer Vision: Machine Perception Through Hierarchical Computation Structures*. Elsevier, 2014.

- [51] David S. Taubman and Michael W. Marcellin. *Jpeg2000: Image Compression Fundamentals, Standards, and Practice*. Kluwer Academic Publishers, 2001.
- [52] Scott E. Umbaugh. *Computer Vision and Image Processing: A Practical Approach Using CVIPTools*. Prentice-Hall, 1998.
- [53] Stefan Van Der Walt et al. “scikit-image: image processing in Python”. In: *PeerJ* 2 (2014), e453.
- [54] James S. Walker. *Fast Fourier Transforms*. Second Edition. CRC Press, 1996.
- [55] Dominic Welsh. *Codes and Cryptography*. Oxford University Press, 1989.
- [56] Joan L. Mitchell William B. Pennebaker. *JPEG Still Image Data Compression Standard*. Van Nostrand Reinhold, 1993.

